

AN AGILE VIEW OF PROCESS

ISE 2019

RAPID SOFTWARE DEVELOPMENT

- Rapid development and delivery is now often the most important requirement for software systems
 - Businesses operate in a fast –changing requirement and it is practically impossible to produce a set of stable software requirements
 - Software has to evolve quickly to reflect changing business needs.
- Rapid software development
 - Specification, design and implementation are inter-leaved
 - System is developed as a series of versions with stakeholders involved in version evaluation
 - User interfaces are often developed using an IDE and graphical toolset.



AGILE METHODS

- Dissatisfaction with the overheads involved in software design methods of the 1980s and 1990s led to the creation of agile methods. These methods:
 - Focus on the code rather than the design
 - Are based on an iterative approach to software development
 - Are intended to deliver working software quickly and evolve this quickly to meet changing requirements.
- The aim of agile methods is to reduce overheads in the software process (e.g. by limiting documentation) and to be able to respond quickly to changing requirements without excessive rework.



AGILE

- AGILE IS A SET OF VALUES AND PRINCIPLES
- Agile is really a collection of beliefs that teams can use for making decision about how to do the work of developing software.



- Effective (rapid and adaptive) response to change
- Effective communication among all stakeholders
- Drawing the customer onto the team
- Organizing a team so that it is in control of the work performed

Yielding ...

- Rapid, incremental delivery of software



AGILE DEVELOPMENT PROCESS

In software development term agile is means the ability to respond to changes , changes from Requirements ,Technology and People.

- Its an iterative and incremental process.
- Direct collaboration with the customers.
- Delivers multiple software increments.
- Engineering actions are carried out by cross out by cross functional teams.
- Effective (rapid and adaptive) response to change
- Effective communication among all stakeholders
- Drawing the customer onto the team
- Organizing a team so that it is in control of the work performed



AGILE MANIFESTO

- Agile manifesto is only 68 words and very simply says that we can develop software better by **valuing** the items on the left side of the list more than the items on the right side.



- Individuals and Interactions

Value

- Processes and tools

- Working software

more than

- Comprehensive documentation

- Customer collaboration

- Contract negotiation

- Responding to change

- Following a plan



- In addition to the values of the manifesto, there are 12 principles that support the values.
- Once again the principles are very general and are less about telling you what to do than they're about giving you the ability to make a good decision in a particular situation.



AGILITY PRINCIPLES

1. Our highest priority is **to satisfy the customer** through early and continuous delivery of valuable software.
2. **Welcome changing requirements**, even late in development. Agile processes harness change for the customer's competitive advantage.
3. **Deliver working software frequently**, from a couple of weeks to a couple of months, with a preference to the shorter timescale.
4. Business people and developers must **work together** daily throughout the project.
5. Build projects around **motivated individuals**. Give them the environment and support they need, and **trust** them to get the job done.
6. The most efficient and effective method of conveying information to and within a development team is **face-to-face conversation**.



AGILITY PRINCIPLES

7. **Working software** is the primary **measure of progress**.
8. Agile processes **promote sustainable development**. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
9. **Continuous attention** to technical excellence and good design enhances agility.
10. **Simplicity** – the art of maximizing the amount of work not done – is essential.
11. The best architectures, requirements, and designs emerge from **self-organizing teams**.
12. At regular intervals, the **team reflects on how to become more effective**, then tunes and adjusts its behavior accordingly.

PROBLEMS WITH AGILE METHODS

- It can be difficult to keep the interest of customers who are involved in the process.
- Team members may be unsuited to the intense involvement that characterizes agile methods.
- Prioritising changes can be difficult where there are multiple stakeholders.
- Maintaining simplicity requires extra work.
- Contracts may be a problem as with other approaches to iterative development.



AGILE METHODS AND SOFTWARE MAINTENANCE

- Most organizations spend more on maintaining existing software than they do on new software development. So, if agile methods are to be successful, they have to support maintenance as well as original development.
- Two key issues:
 - Are systems that are developed using an agile approach maintainable, given the emphasis in the development process of minimizing formal documentation?
 - Can agile methods be used effectively for evolving a system in response to customer change requests?
- Problems may arise if original development team cannot be maintained.



PLAN-DRIVEN AND AGILE DEVELOPMENT

○ Plan-driven development

- A plan-driven approach to software engineering is based around separate development stages with the outputs to be produced at each of these stages planned in advance.
- Not necessarily waterfall model – plan-driven, incremental development is possible
- Iteration occurs within activities.

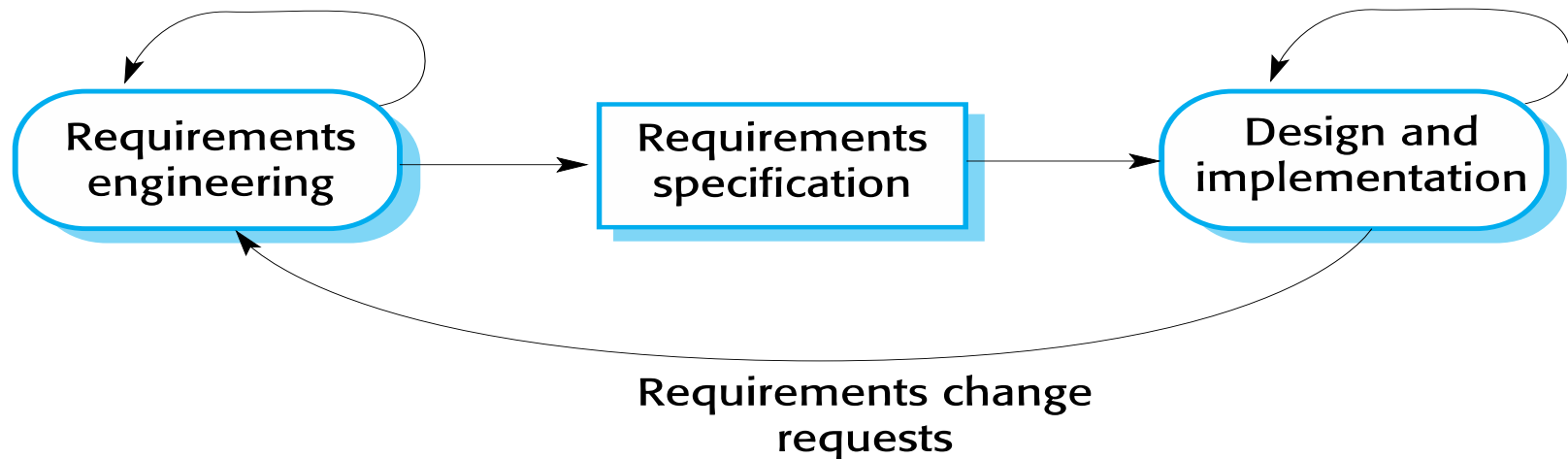
○ Agile development

- Specification, design, implementation and testing are interleaved and the outputs from the development process are decided through a process of negotiation during the software development process.

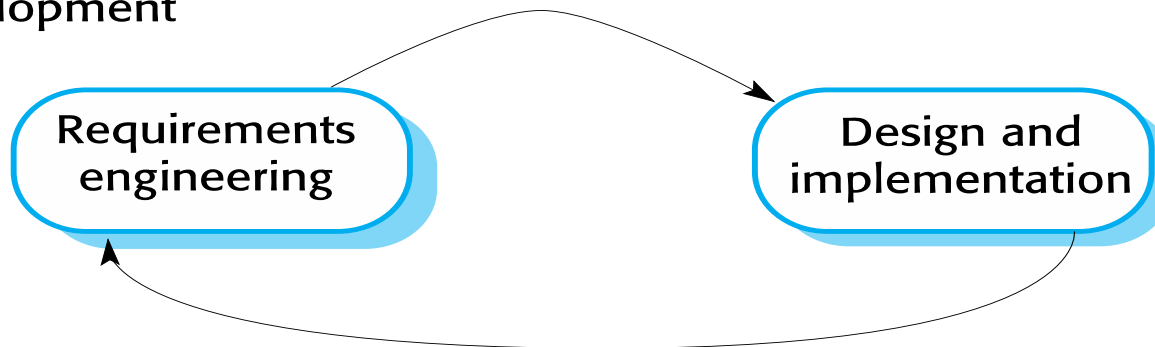


PLAN-DRIVEN AND AGILE DEVELOPMENT

Plan-based development



Agile development



TECHNICAL, HUMAN, ORGANIZATIONAL ISSUES

- What type of system is being developed?
 - Plan-driven approaches may be required for systems that require a lot of analysis before implementation (e.g. real-time system with complex timing requirements).
- What is the expected system lifetime?
 - Long-lifetime systems may require more design documentation to communicate the original intentions of the system developers to the support team.
- What technologies are available to support system development?
 - Agile methods rely on good tools to keep track of an evolving design
- How is the development team organized?
 - If the development team is distributed or if part of the development is being outsourced, then you may need to develop design documents to communicate across the development teams.

TECHNICAL, HUMAN, ORGANIZATIONAL ISSUES

- Are there cultural or organizational issues that may affect the system development?
 - Traditional engineering organizations have a culture of plan-based development, as this is the norm in engineering.
- How good are the designers and programmers in the development team?
 - It is sometimes argued that agile methods require higher skill levels than plan-based approaches in which programmers simply translate a detailed design into code



1. EXTREME PROGRAMMING (XP)

- Extreme Programming (XP) is an agile software development framework that aims to produce higher quality software, and higher quality of life for the development team.
- XP is a lightweight , efficient, low-risk, flexible, predictable, scientific, & fun way to develop software
- Small to medium sized teams that work under vague (uncertain) rapidly changing requirements.



EXTREME PROGRAMMING (XP)

- The most widely used agile process, originally proposed by Kent Beck
- XP Planning
 - Begins with the creation of “**user stories**”
 - Agile team assesses each story and assigns a **cost**
 - Stories are grouped to for a **deliverable increment**
 - A **commitment** is made on delivery date
 - After the first increment “**project velocity**” is used to help define subsequent delivery dates for other increments



VALUES OF XP

- Values

The five values of XP are communication, simplicity, feedback, courage, and respect and are described in more detail below.

- Communication

- Software development is inherently a team sport that relies on communication to transfer knowledge from one team member to everyone else on the team. XP stresses the importance of the appropriate kind of communication - face to face discussion with the aid of a white board or other drawing mechanism.



○ **Simplicity**

Simplicity means “what is the simplest thing that will work?” The purpose of this is to avoid waste and do only absolutely necessary things such as keep the design of the system as simple as possible so that it is easier to maintain, support, and revise. Simplicity also means address only the requirements that you know about; don’t try to predict the future.

○ **Feedback**

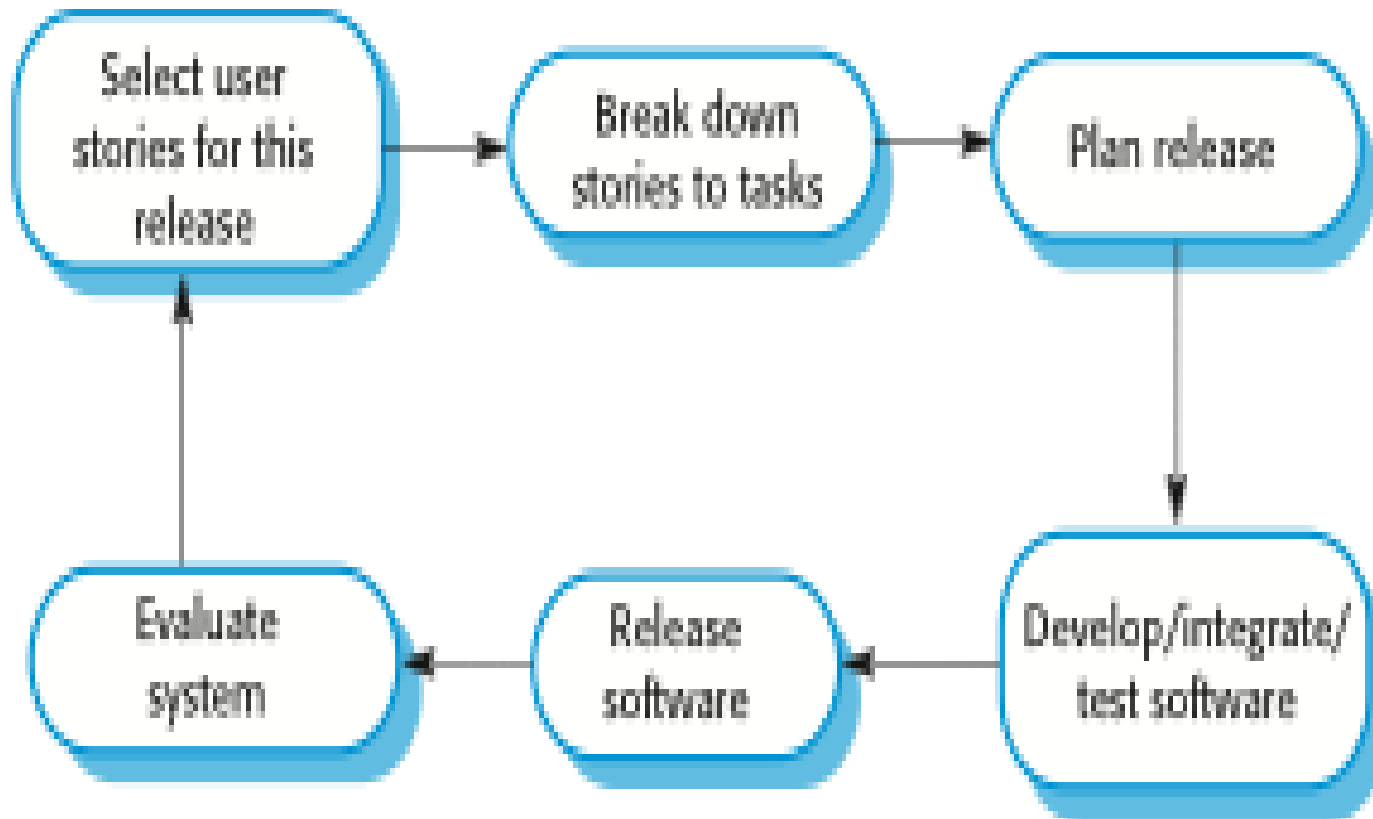
Through constant feedback about their previous efforts, teams can identify areas for improvement and revise their practices. Feedback also supports simple design. Your team builds something, gathers feedback on your design and implementation, and then adjust your product going forward.



Courage

- Kent Beck defined courage as “effective action in the face of fear.” This definition shows a preference for action based on other principles so that the results aren’t harmful to the team. You need courage to raise organizational issues that reduce your team’s effectiveness. You need courage to stop doing something that doesn’t work and try something else. You need courage to accept and act on feedback, even when it’s difficult to accept.

- More focus on software development



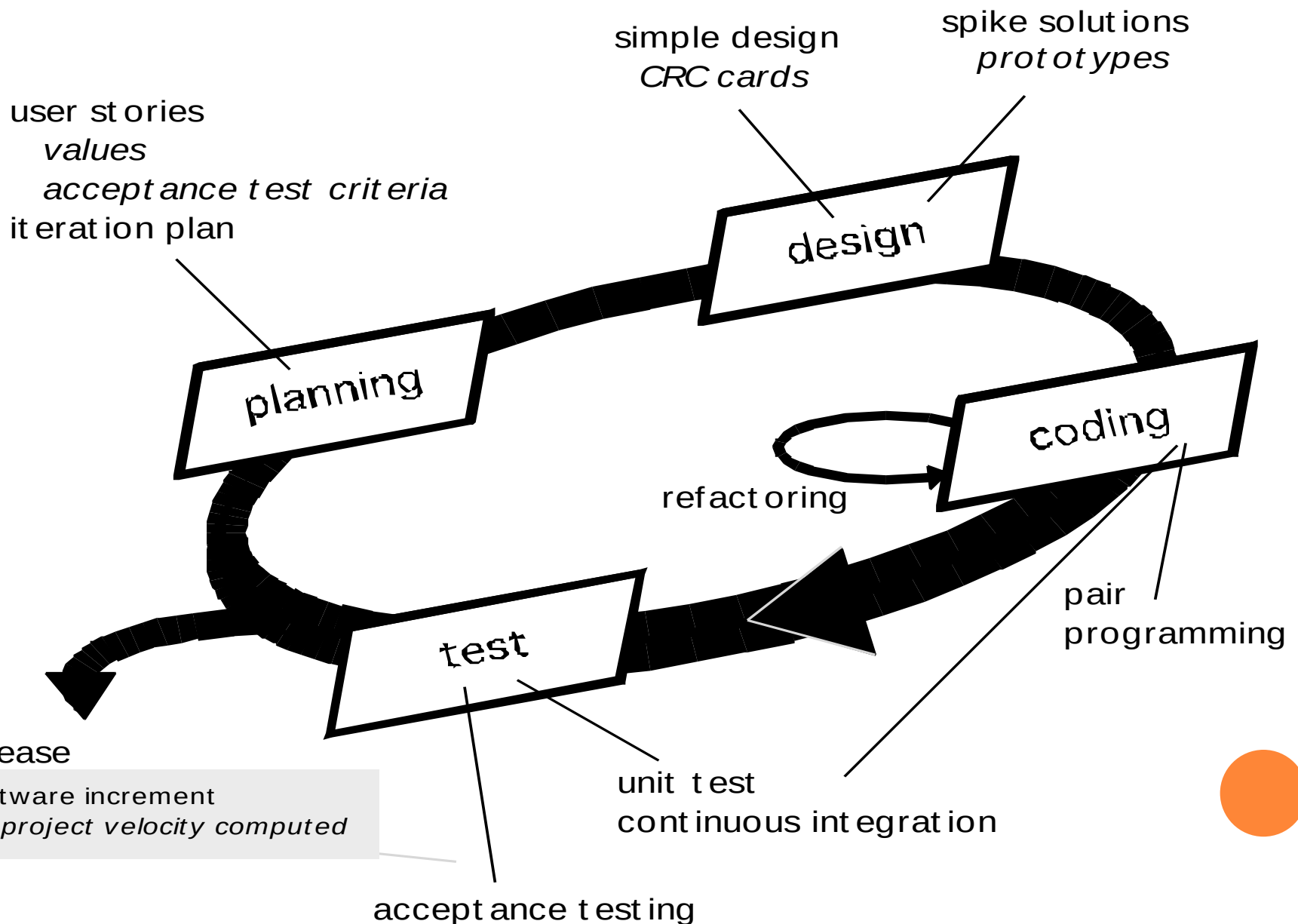
EXTREME PROGRAMMING PRACTICES

- **Principle or practice**
- **Incremental planning**
- Requirements are recorded on story cards and the stories to be included in a release are determined by the time available and their relative priority. The developers break these stories into development 'Tasks'.
- **Small releases**
- The minimal useful set of functionality that provides business value is developed first. Releases of the system are frequent and incrementally add functionality to the first release.
- **Simple design**
- Enough design is carried out to meet the current requirements and no more.
- **Test-first development**
- An automated unit test framework is used to write tests for a new piece of functionality before that functionality itself is implemented.
- **Refactoring**
- All developers are expected to refactor the code continuously as soon as possible code improvements are found. This keeps the code simple and maintainable.

EXTREME PROGRAMMING PRACTICES

- **Pair programming**
- Developers work in pairs, checking each other's work and providing the support to always do a good job.
- **Collective ownership**
- The pairs of developers work on all areas of the system, so that no islands of expertise develop and all the developers take responsibility for all of the code. Anyone can change anything.
- **Continuous integration**
- As soon as the work on a task is complete, it is integrated into the whole system. After any such integration, all the unit tests in the system must pass.
- **Sustainable pace**
- Large amounts of overtime are not considered acceptable as the net effect is often to reduce code quality and medium term productivity
- **On-site customer**
- A representative of the end-user of the system (the customer) should be available full time for the use of the XP team. In an extreme programming process, the customer is a member of the development team and is responsible for bringing system requirements to the team for implementation.

EXTREME PROGRAMMING (XP)



- Spike solution:
- Is a very simple program to explore potential solutions. Difficult designs should be modeled using prototype.
- Refactoring :
- Improves the internal structure of the code but external behavior not affected.



2. SCRUM

- Scum is agile development process for building complex and sustainable products.
- 3 roles
 1. Product Owner
 2. Scrum Master
 3. Scrum Team



PRODUCT OWNER

- The product owner creates a product backlog(essentially a wishlist of tasks that need to be prioritized in project).
- Decide Release date and content.
- Prioritize features according to market value.
- Accept or Reject work results.



SCRUM MASTER

- The scrum master conducts daily meetings.
- Ensures that the team is fully functional and productive.
- Shield the team from external interfaces.
- Responsible for management of the project.



SCRUM TEAM

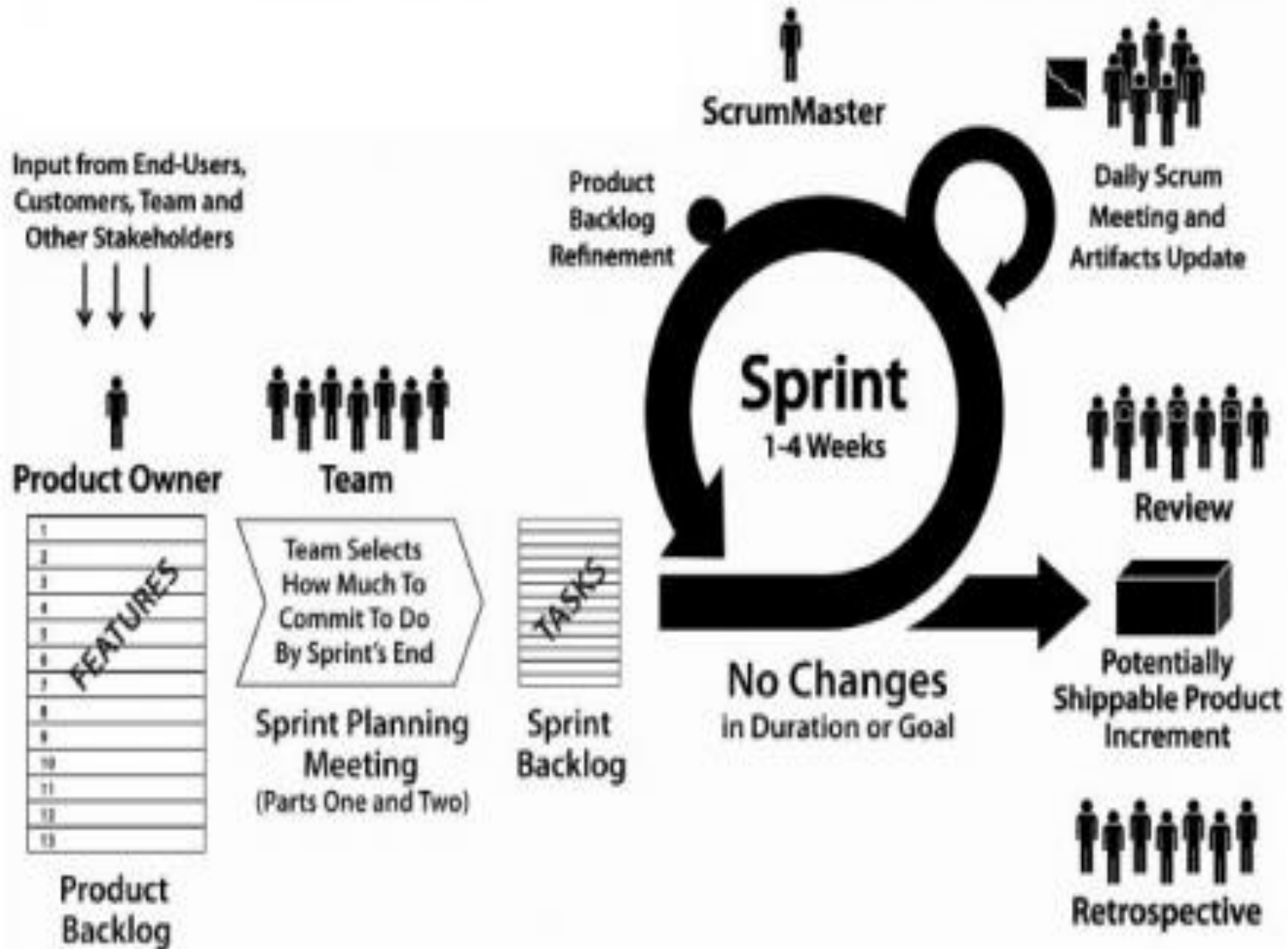
- Typically 5 to 10 people.
- Teams are cross functional and self organizing.
- Scrum team determines which items and in what order are to be executed.



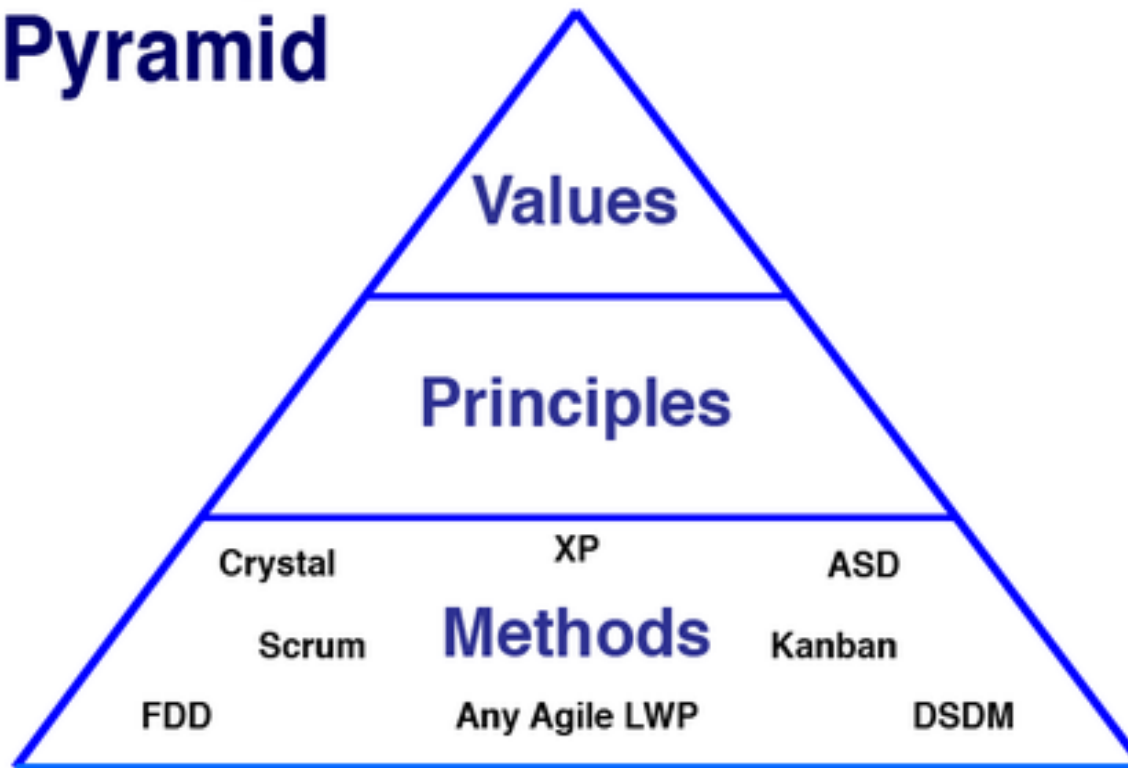
SCRUM

- Originally proposed by Schwaber and Beedle
- Scrum—distinguishing features
 - Development work is partitioned into “packets”
 - Testing and documentation are on-going as the product is constructed
 - Work occurs in “sprints” and is derived from a “backlog” of existing requirements
 - Meetings are very short and sometimes conducted without chairs
 - “demos” are delivered to the customer with the time-box allocated





The Agile Pyramid



Attribution-NonCommercial-ShareAlike 4.0 International



Copyright AgileLion Institute

